

8 – 2 Journal: Portfolio Reflection

Crystal Schippers

Southern New Hampshire University

CS 405: Secure Coding

Professor: Olga Mill

Date: August 23, 2026

Reflection: Secure Coding and Security Policies

Throughout this course, I have learned that security should be treated as an ongoing part of software development rather than something addressed after a vulnerability is discovered. One of the most important lessons for me was the importance of adopting a secure coding standard and not leaving security to the end. Using standards such as the SEI CERT C++ Coding Standard gives developers specific practices to follow when writing code, including validating input, using proper memory management, handling exceptions, and following the principle of least privilege. Incorporating these practices during development can help identify vulnerabilities earlier, when they are generally easier and less expensive to correct.

Another important lesson was the need to evaluate risk and consider the cost and benefits of mitigation. Not every vulnerability has the same level of severity or likelihood, so organizations should prioritize issues based on the potential impact on confidentiality, integrity, and availability. For example, a vulnerability that could expose sensitive customer information should receive a higher priority than a low-impact coding issue. Tools such as static code analysis and unit testing can help developers identify problems early and determine which issues require immediate attention. This approach allows organizations to use their resources effectively while still maintaining a strong security posture.

The concept of zero trust also changed how I think about security. The idea of "no one is safe" means that users, devices, applications, and network connections should not automatically be trusted simply because they are inside an organization's environment. Access should be verified and limited according to what is actually required. This connects closely with the principle of least privilege and the Triple-A framework of authentication, authorization, and accounting. Implementing zero trust can reduce the potential damage if an account or device is

compromised because an attacker would not automatically receive unrestricted access to the entire system.

Finally, I learned that security policies must be implemented, maintained, and continuously improved. A security policy is only useful if developers understand it and consistently apply it. Organizations should provide clear coding standards, automate security checks throughout the DevSecOps pipeline, perform regular testing, and update policies as new threats emerge. Practices such as static analysis, unit testing, vulnerability scanning, and code reviews can help enforce these policies throughout the development lifecycle. Overall, this course showed me that effective security requires multiple layers to work together. By combining secure coding standards, risk assessment, zero trust, automation, and defense in depth, developers can build security into applications from the beginning instead of trying to fix security problems after they occur.